Programming and Data Structures

500+ One-Liners for Rapid Revision

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

Programming and Data Structures - One-Liners Rapid Revision 4

- C Language Basics 4
- Data Types and Variables 4
- Operators and Expressions 4
- Control Flow 5
- Functions 5
- Pointers 5
- Arrays and Strings 6
- Structures and Unions 6
- Dynamic Memory Allocation 6
- File Handling and I/O 6
- Preprocessor 7

Recursion 8

Abstract Data Type 8

Arrays as a Data Structure 8

Stacks 8

Queues 9

Linked Lists 9

Tree Terminology 10

Binary Trees 10

Binary Search Trees 10

Balanced Trees and Heaps 10

Graphs 11

Hashing 11

Complexity Summary 11

C Programming - Deeper Nuances 13

Data Structures - Deeper Nuances 13

Hashing and Complexity - Deeper Nuances 14

C Output-Prediction Essentials 15

Pointer and Array Output Nuances 15

String Function Nuances 15

Recursion Tracing Nuances 16

Stack/Queue Application Nuances 16

Tree Traversal Nuances 16

BST and Heap Sequence Nuances 16

Graph and Hashing Nuances 17

Final Rapid-Fire Facts 17

Extended C Concept Drills 18

Extended Data-Structure Drills 18

Extended Tree Drills 19

Extended Graph and Hashing Drills 19

Final Complexity Cheat Drills 20

Supplementary Exam One-Liners 20

Programming and Data Structures - One-Liners Rapid Revision

C Language Basics

- A C program build has **4** stages: **preprocessing**, **compilation**, **assembly**, and **linking**.
- Execution of every C program begins at **main()**, which returns type **int** to the OS.
- The successful-exit return code from main is **0** (also written `EXIT_SUCCESS`).
- C has **6** token classes: **keywords**, **identifiers**, **constants**, **string literals**, **operators**, and **punctuators**.
- C89/C90 defines **32** reserved keywords, while C99 adds **5** more for a total of **37**.
- Identifiers may begin with only **2** kinds of characters: a **letter** or an **underscore**.
- C is **case-sensitive**, so **int** and **Int** are treated as **different** tokens.
- `#include <...>` searches **system** directories while `#include "..."` searches the **current** directory first.
- C supports **2** comment styles: **block /* */** and **line //** (C99).
- The tool stage that reports an "**undefined reference**" error is the **linker**, not the compiler.
- The preprocessor produces a **.i** file, the compiler a **.s** file, the assembler a **.o** file, and the linker the **executable**.
- The single output stage of a normal C program for input is provided by **scanf** and for output by **printf**.
- The **address-of (&)** operator is required for each variable in a **scanf** call.

Data Types and Variables

- C has **5** basic data types: **char**, **int**, **float**, **double**, and **void**.
- Typical 64-bit sizes are: **char=1**, **short=2**, **int=4**, **double=8** bytes.
- `sizeof(char)` is guaranteed to equal **1** by the C standard.
- The `sizeof` construct is a **compile-time operator** (not a function) whose result type is **size_t**.
- A signed n-bit integer ranges from **-2^(n-1)** to **2^(n-1) - 1**.
- A 32-bit signed int ranges from **-2,147,483,648** to **2,147,483,647**.
- An unsigned n-bit value ranges from **0** to **2^n - 1**.
- IEEE 754 float is **32-bit** (~**7** digits) while double is **64-bit** (~**15** digits).
- C has **4** storage classes: **auto**, **register**, **static**, and **extern**.
- The default storage class for local variables is **auto**, stored on the **stack**.
- A **register** variable cannot have its **address** taken with `&`.
- A **static** local variable is initialised **once** and keeps its value across **calls**.
- **Global** and **static** variables default-initialise to **0**, but **auto** locals hold **garbage**.
- The **extern** keyword declares a variable defined **elsewhere** and allocates **no** storage.
- C has **2** main type qualifiers: **const** (read-only) and **volatile** (no caching).
- A **volatile** qualifier is essential for **memory-mapped registers** and **interrupt**-modified variables.
- An `enum` starts numbering at **0** and increments by **1** unless overridden.
- In `enum {A,B=5,C}` the values are **A=0**, **B=5**, and **C=6**.
- `#define` is handled by the **preprocessor** (no type), while `const` is handled by the **compiler** (typed).
- A **typedef** creates an **alias** for an existing type and does **not** create a new type.
- Integer literal suffixes include **U** (unsigned), **L** (long), and **LL** (long long).

Operators and Expressions

- Integer division `7/2` yields **3** because it **truncates** toward zero.
- The modulo operator `%` works only on **integer** operands and `7%2` gives **1**.

- C has **6** bitwise operators: **& | ^ ~ << >>**.
- To set bit k use **x | (1<<k)**; to clear it use **x & ~(1<<k)**; to toggle it use **x ^ (1<<k)**.
- Left shift by k multiplies by **2^k** and right shift divides by **2^k**.
- Short-circuit `&&` stops at the first **false** and `||` stops at the first **true**.
- The only **three-operand** (ternary) operator in C is **?:**.
- The **comma** operator evaluates left-to-right and yields the **rightmost** value.
- Most binary operators are **left-associative**, but **assignment**, **unary**, and **?:** are **right-associative**.
- The expression `a & b == c` parses as **a & (b==c)** because **==** outranks **&**.
- The expression `1<<2+3` equals **32** because **+** outranks **<<**.
- Usual arithmetic promotion order is **char/short -> int -> unsigned -> long -> float -> double**.
- The expression `5/2*2.0` evaluates to **4.0**, not 5.0, due to left-to-right integer division first.
- `y = x++` gives y the **old** x, while `y = ++x` gives y the **new** x.
- Multiple unsequenced modifications like `i = i++ + ++i` are **undefined behaviour**.

Control Flow

- C has **3** loop constructs: **for**, **while**, and **do-while**.
- The only **exit-controlled** loop in C is **do-while**, which runs at least **1** time.
- **for** and **while** are **entry-controlled** loops with a minimum of **0** executions.
- A `switch` requires **break** to prevent **fall-through** between cases.
- A `switch` expression must be an **integer or char** constant type, never **float** or **string**.
- **break** exits **1** level of loop/switch, while **continue** skips to the **next iteration**.
- The **goto** statement jumps to a **label** within the **same function**.
- Two nested loops of sizes n and m run a total of **n x m** iterations.
- The dangling-else rule binds an `else` to the **nearest unmatched if**.

Functions

- A function **prototype** declares **return type, name, and parameter types** and ends with `;`.
- C passes arguments **by value** by **default**, so the callee gets a **copy**.
- To modify a caller's variable, you must pass a **pointer** (address) and **dereference** it.
- A function returns **at most one** value via **return**, which also **terminates** it.
- A function pointer is declared as **int (*fp)(int,int)** and used for **callbacks** and **jump tables**.
- `int (*fp)(int)` is a **pointer to function**, but `int *fp(int)` is a **function returning int***.

Pointers

- The **&** operator gives an **address** and ***** performs **dereference/indirection**.
- A **NULL** pointer has value **0** and dereferencing it is **undefined behaviour**.
- Adding 1 to an `int*` advances by **4** bytes due to **pointer scaling** (sizeof type).
- Valid pointer operations include **pointer + int**, **pointer - int**, and **pointer - pointer**, but **pointer + pointer** is illegal.
- The difference `p - q` yields the **number of elements** of type **ptrdiff_t**, not bytes.
- An array name **decays** to a pointer to its **first** element, so `a[i]` equals ***(a+i)**.
- Because indexing is commutative, `a[i]` also equals **i[a]**.
- An array name is **not** a modifiable lvalue, so **a++** is illegal.
- A **void *** is a **generic** pointer that **cannot** be dereferenced without a **cast**.
- The function `malloc` returns a **void *** pointer.

- A **dangling** pointer references **freed/out-of-scope** memory, while a **leak** is memory never **freed**.
- Returning the address of a **local** variable produces a **dangling** pointer.
- The const placement `const int *p` means **pointer to const data**, while `int *const p` means **const pointer**.
- **Double indirection** with `int **pp2` stores the address of a **pointer** and reaches data via **two dereferences**.

Arrays and Strings

- Array indices in C run from **0** to **n-1**, and C performs **no** bounds checking.
- A 2D array `a[r][c]` is stored in **row-major** order as an **array of arrays**.
- The row-major address of `a[i][j]` is **B + (i*C + j)*s** where C is the column count.
- An array passed to a function **decays** to a pointer, losing its **size** information.
- A C string is a **char array** terminated by **'\0'** (ASCII **0**).
- For `"Hi"`, **strlen** returns **2** but **sizeof** returns **3** (counts the '\0').
- The standard string functions include **strlen**, **strcpy**, **strcat**, **strcmp**, **strchr**, and **strstr**.
- `strcmp` returns **0** when strings are **equal**, and negative/positive otherwise.
- `strlen` runs in **O(n)** time scanning to the **'\0'**.
- An **array of pointers** `char *arr[]` is ideal for **ragged** string tables like **argv**.

Structures and Unions

- A **struct** groups **different**-typed members, accessed with **.** for objects and **->** for pointers.
- The expression `p->x` is shorthand for **(*p).x**.
- A **self-referential** struct contains a pointer to its **own type**, the basis of **linked lists** and **trees**.
- A **union** stores all members in **shared** memory and its size equals the **largest** member.
- In a union, only **1** member is valid at a time (the **last written** one).
- Struct size may exceed the member-sum due to **padding** for **alignment**.
- **Bit-fields** let a struct member occupy a specified number of **bits**.

Dynamic Memory Allocation

- A process image has **4** regions: **text/code**, **data**, **heap**, and **stack**.
- The **heap** grows **up** and the **stack** grows **down** (typical convention).
- The **4** dynamic memory functions are **malloc**, **calloc**, **realloc**, and **free**.
- **malloc** leaves memory **uninitialised** while **calloc zero**-initialises it.
- `calloc(n,size)` takes **2** arguments while `malloc(size)` takes **1**.
- All allocators return a **void *** (or **NULL** on failure) and live in **<stdlib.h>**.
- The **3** classic heap errors are **memory leak**, **double free**, and **use-after-free**.
- After **realloc**, the **old** pointer may be **invalid**, so use the **returned** pointer.

File Handling and I/O

- Common format specifiers are **%d** int, **%f** float, **%c** char, **%s** string, **%x** hex, and **%p** pointer.
- `printf` returns the number of **characters printed** and `scanf` the number of **items read**.
- The **4** fopen base modes are **"r"**, **"w"**, **"a"**, plus **"+"** and **"b"** variants.
- Mode **"r" fails** if the file is missing, while **"w" creates/truncates** it.
- `fopen` returns a **FILE *** or **NULL** on failure.
- The unsafe input function **gets** is removed; the safe replacement is **fgets**.
- **EOF** is a macro commonly equal to **-1** that signals end of file.

- In `main(int argc, char *argv[])`, **argc** is always **>=1** because **argv[0]** holds the program name.
- The element **argv[argc]** is always **NULL**.

Preprocessor

- The preprocessor supports **object-like** macros like `#define PI 3.14` and **function-like** macros like `#define MAX(a,b)`.
- Macros are expanded **textually** with **no** type checking and no **call overhead**.
- Conditional compilation uses **#ifdef**, **#ifndef**, and **#endif**.
- An **include guard** (or **#pragma once**) prevents **double inclusion** of a header.
- The macro `#define SQ(x) x*x` makes `SQ(2+3)` expand to **11**, showing why arguments need **parentheses**.

Recursion

- A recursive function needs **2** parts: a **base case** and a **recursive case**.
- A missing/incorrect base case causes **infinite recursion** leading to **stack overflow**.
- Each recursive call pushes an **activation record** (parameters, locals, return address) on the **call stack**.
- Recursion uses **O(depth)** stack space where depth is the **recursion depth**.
- Recursion has **2** forms by structure: **direct** (calls itself) and **indirect/mutual** (f calls g calls f).
- **Tail** recursion has the recursive call as the **last** operation and can run in **O(1)** stack via **tail-call optimisation**.
- Factorial `n!` runs in **O(n)** time with base case **0! = 1**.
- Fast exponentiation computes `x^n` in **O(log n)** by **squaring**.
- Euclid's GCD `gcd(a,b)=gcd(b,a%b)` runs in **O(log min(a,b))** with base **gcd(a,0)=a**.
- Naive Fibonacci runs in **O(2^n)** time but memoised Fibonacci runs in **O(n)**.
- Tower of Hanoi needs a minimum of **2^n - 1** moves, e.g. **7** moves for 3 disks.
- The Hanoi recurrence is **T(n)=2T(n-1)+1**.
- Naive Fibonacci makes **2*F(n+1) - 1** total calls, e.g. **15** calls for fib(5).
- Recursion versus iteration trades **O(depth) space** and call overhead for **cleaner** self-similar code.

Abstract Data Type

- An **ADT** defines **values and operations** (the **what**), hiding the **how**.
- The **3** levels of abstraction are **ADT**, **data structure**, and **implementation**.
- **Stack**, **Queue**, and **List** are **ADTs**, while **array** and **linked list** are **data structures**.

Arrays as a Data Structure

- A **static** array has **fixed** compile-time size on the **stack**; a **dynamic** array is sized at **runtime** on the **heap**.
- Array random access by index costs **O(1)** via address arithmetic.
- Searching an unsorted array costs **O(n)**; a sorted array with binary search costs **O(log n)**.
- Inserting/deleting at an arbitrary array position costs **O(n)** due to **shifting**.
- Inserting at index i shifts **n-i** elements right; deleting shifts **n-i-1** left.
- The 1D address of `a[i]` with lower bound L is **B + (i-L)*s**.

Stacks

- A **stack** follows **LIFO** (Last-In-First-Out) and operates only at the **top**.
- A stack supports **5** operations: **push**, **pop**, **peek**, **isEmpty**, and **isFull**.
- All core stack operations run in **O(1)**.
- In an array stack, **overflow** occurs at top=**capacity-1** and **underflow** at top=**-1**.
- A linked-list stack pushes/pops at the **head** and avoids fixed **overflow**.
- Stacks power **5** classic applications: **function call stack**, **balanced symbols**, **infix-to-postfix**, **postfix evaluation**, and **undo/backtracking**.
- Balanced-bracket checking is balanced iff the stack is **empty at end** and never pops **empty**.
- Postfix notation is also called **Reverse Polish Notation (RPN)** and needs **no** parentheses.
- Evaluating postfix pops **2** operands per operator and pushes the **result**.

Queues

- A **queue** follows **FIFO** (First-In-First-Out): insert at **rear**, remove from **front**.
- A queue supports **enqueue**, **dequeue**, **front**, and **rear**, all **O(1)**.
- A linear array queue suffers **false overflow** because vacated **front** cells cannot be reused.
- A **circular** queue wraps indices with **modulo**: `rear=(rear+1)%size`.
- A circular queue is **full** when **(rear+1)%size==front** and **empty** when **front==rear**.
- A capacity-N circular queue (one-slot-unused) usefully holds **N-1** elements.
- A **deque** allows insertion/deletion at **both** ends and has **input-restricted** and **output-restricted** forms.
- A **priority queue** dequeues the **highest-priority** element and is built on a **binary heap** with **O(log n)** operations.
- Queues power **3** applications: **scheduling**, **BFS**, and **buffering**.

Linked Lists

- A linked-list node stores **data** plus one or more **pointers** via a **self-referential** struct.
- In a singly linked list, traversal and search cost **O(n)** while head insert/delete cost **O(1)**.
- Linked-list insertion needs only **pointer rewiring** with **no** shifting.
- Iterative list reversal uses **3** pointers (prev, curr, next) in **O(n)** time and **O(1)** space.
- The **middle** of a list is found with **slow/fast (tortoise-hare)** pointers in one pass.
- **Floyd's cycle detection** uses **slow(+1)** and **fast(+2)** pointers, running in **O(n)** time and **O(1)** space.
- A **doubly** linked list stores **prev and next**, enabling **O(1)** node deletion and **bidirectional** traversal.
- A **circular** linked list links the last node back to the **first**, useful for **round-robin** scheduling.
- An array gives **O(1)** access while a linked list gives **O(n)** access but **O(1)** front insert.
- Linked lists have **no** random access, so reaching the k-th node costs **O(k)**.
- Singly-list deletion needs the **previous** node, otherwise it is **O(n)**.
- Linked lists represent **polynomials** as **coefficient + exponent + next** terms.

Tree Terminology

- A tree node's **depth** is the edge count from the **root**, and its **height** is edges to the deepest **leaf**.
- The height of a single node (leaf) is **0** and the height of an empty tree is **-1**.
- A node's **degree** equals its number of **children**.
- A tree with n nodes has exactly **n-1** edges.
- A **leaf** has **0** children while the **root** has **no** parent.

Binary Trees

- A binary tree node has **at most 2** children (left and right).
- The maximum nodes at level i (root level 0) is **2^i**.
- The maximum nodes in a binary tree of height h is **2^(h+1) - 1**.
- A binary tree with n nodes has exactly **n+1** NULL pointers.
- There are **5** binary tree types: **full**, **complete**, **perfect**, **skewed**, and **balanced**.
- A **full/strict** binary tree has every node with **0 or 2** children.
- A **complete** binary tree fills all levels except the last, which fills **left to right**.
- A **perfect** binary tree has all leaves at the **same** level with **2^(h+1)-1** nodes.
- A **skewed** binary tree has **one** child per node, behaving like a **linked list** of height **n-1**.
- In a full binary tree, leaves = **internal-with-two-children + 1**.
- There are **4** binary tree traversals: **inorder**, **preorder**, **postorder**, and **level-order**.
- **Inorder** visits **Left, Root, Right** and gives **sorted** output for a BST.
- **Preorder** visits **Root, Left, Right** and places the **root first**.
- **Postorder** visits **Left, Right, Root** and places the **root last**, used to **free** a tree.
- **Level-order** traversal uses a **queue** and runs in **O(n)**.
- All four traversals run in **O(n)** time.
- Unique tree reconstruction needs **inorder** plus either **preorder** or **postorder**.
- **Preorder + postorder** alone **cannot** uniquely rebuild a general binary tree.
- In an **expression tree**, inorder gives **infix**, postorder gives **postfix**, and preorder gives **prefix**.
- The number of distinct binary trees with n nodes is the **Catalan number (2n)!/((n+1)! n!)**.
- The array representation uses left=**2i+1**, right=**2i+2**, parent=**(i-1)/2** (0-based).

Binary Search Trees

- A **BST** keeps **left < node < right** for every node.
- BST search, insert, and delete each cost **O(h)**: **O(log n)** balanced, **O(n)** skewed.
- BST deletion has **3** cases: **leaf**, **one child**, and **two children**.
- A two-child BST node is replaced by its **inorder successor** (right-subtree min) or **predecessor** (left-subtree max).
- The BST **minimum** is the **leftmost** node and the **maximum** is the **rightmost**.
- An **inorder** traversal of a BST outputs keys in **ascending** order.
- Inserting **sorted** data into a plain BST creates a **skewed** tree with **O(n)** operations.

Balanced Trees and Heaps

- An **AVL** tree keeps balance factor **height(left)-height(right)** in **{-1,0,+1}**.

- AVL trees use **4** rotation cases: **LL**, **RR**, **LR**, and **RL**.
- **LL/RR** need a **single** rotation while **LR/RL** need a **double** rotation.
- AVL trees guarantee height **O(log n)** and thus **O(log n)** operations.
- A **red-black** tree maintains balance with **5** properties including **root black** and **no two adjacent reds**.
- A **binary heap** is a **complete** binary tree; max-heap parent **>=** children, min-heap parent **<=** children.
- Heap insert uses **sift-up** and delete-root uses **sift-down (heapify)**, both **O(log n)**.
- Heap peek (min/max) costs **O(1)** at the **root**.
- **Build-heap** from an array costs **O(n)**, not O(n log n).
- **Heap sort** runs in **O(n log n)** time, **O(1)** extra space, and is **not stable**.
- A heap is **not** fully sorted; only the **root** is the extreme element.

Graphs

- A graph **G=(V,E)** has **vertices** and **edges**, classified as **directed/undirected** and **weighted/unweighted**.
- The sum of all vertex degrees equals **2*|E|** (Handshaking Lemma).
- A complete undirected graph **Kn** has **n(n-1)/2** edges.
- A **tree** on n vertices is connected and acyclic with exactly **n-1** edges.
- A graph is **bipartite** iff it contains **no odd-length cycle**.
- A **DAG** is a **directed acyclic** graph.
- An **adjacency matrix** uses **O(V^2)** space with **O(1)** edge lookup.
- An **adjacency list** uses **O(V+E)** space, best for **sparse** graphs.
- **BFS** uses a **queue** and finds shortest paths in **unweighted** graphs.
- **DFS** uses **recursion/stack**, both traversals costing **O(V+E)**.
- A **back edge** found during DFS indicates a **cycle**.

Hashing

- A **hash table** targets **O(1)** average lookup/insert/delete via a **hash function**.
- The **division** hash method computes **h(k)=k mod m**.
- The **load factor** is **alpha = n/m** (entries / buckets).
- **Separate chaining** stores collisions in a **linked list** and allows alpha **> 1**.
- **Open addressing** stores entries in the **table itself** and requires alpha **< 1**.
- **Linear probing** uses **(h+i) mod m** and suffers **primary clustering**.
- **Quadratic probing** uses **(h+c1 i+c2 i^2) mod m** and suffers **secondary clustering**.
- **Double hashing** uses **(h1+i h2) mod m** and avoids **clustering**.
- **Rehashing** builds a **larger** (often **double**) table when the load factor exceeds a threshold, costing **O(n)**.
- Hash table average operations are **O(1)** but worst-case are **O(n)**.

Complexity Summary

- Array access is **O(1)** but array insert/delete is **O(n)**.
- Linked list access is **O(n)** but head insert/delete is **O(1)**.
- Stack and queue push/pop/enqueue/dequeue are all **O(1)**.
- Balanced BST and AVL operations are **O(log n)**; skewed BST worst case is **O(n)**.
- Heap insert/extract are **O(log n)** and peek is **O(1)**.
- Hashing average lookup is **O(1)** and worst is **O(n)**.

- The top exam traps include **sizeof array vs pointer**, **strlen vs sizeof**, **build-heap O(n)**, and **circular queue capacity N-1**.

C Programming - Deeper Nuances

- The macro `__FILE__`, `__LINE__`, `__DATE__`, and `__TIME__` are **4** predefined preprocessor macros.
- A string literal `"abc"` has type **char[4]** and is stored in **read-only** memory.
- Modifying a string literal via `char *p="abc"; p[0]='x';` is **undefined behaviour**.
- The `static` keyword has **2** meanings: **persistent** storage (locals) and **internal linkage** (file scope).
- The **ternary** operator `a?b:c` evaluates exactly **1** of the two branches.
- A **dangling-else** ambiguity is resolved by attaching else to the **nearest** if.
- The expression `sizeof(int)` typically returns **4** on modern systems.
- The result of `sizeof` is unsigned and of type **size_t**.
- The `%` operator result sign follows the **dividend** in C99 (`-7%3` = **-1**).
- The expression `'A'` has type **int** in C with value **65**.
- The null character `'\0'` has integer value **0** and ends every **C string**.
- A `do-while` loop body executes a minimum of **1** time before the test.
- A `goto` cannot jump **into** a different **function**.
- The conditional operator has **lower** precedence than **arithmetic** but **higher** than **assignment**.
- The increment ++ and decrement `--` operators have **2** forms: **prefix** and **postfix**.
- An array of n elements declared `int a[n]` with constant n has **n** valid indices, 0 to **n-1**.
- A multi-dimensional array `a[2][3]` has **6** total elements.
- The address `&a[0]` equals **a** for an array a (array decays to that pointer).
- A **function-like macro** avoids **call overhead** but lacks **type safety**.
- The keyword used to prevent compiler optimisation of a variable is **volatile**.
- The keyword to give a local persistent lifetime is **static**.
- A pointer subtraction is valid only within the **same array**; otherwise it is **undefined**.
- The `realloc(NULL, size)` call behaves like **malloc(size)**.
- The `realloc(ptr, 0)` call behaves like **free(ptr)** (implementation-defined).
- The `free(NULL)` call is **safe** and does **nothing**.
- A `char` may be **signed** or **unsigned** by default, which is **implementation-defined**.
- The number of bytes in `"Hello\0World"` for sizeof is **12** (counts both halves and final '\0').
- A 2D array name a in `int a[3][4]` has type **pointer to int[4]**.
- The expression `*(*(a+i)+j)` is equivalent to **a[i][j]**.
- An uninitialised global pointer is initialised to **NULL** (value 0).
- An uninitialised local pointer is a **wild** pointer holding **garbage**.
- The maximum value of an `unsigned char` is **255**.
- The wraparound of `unsigned` arithmetic is modulo **2^n** and is **well-defined**.
- Signed integer overflow in C is **undefined behaviour**.
- A `const` pointer to `const` data is written **const int *const p**.
- The format specifier for a `double` in scanf is **%lf** but in printf is **%f**.
- The format specifier for `size_t` is **%zu**.
- The escape sequence `\t` is a **tab** and `\n` is a **newline**.
- An octal literal in C starts with **0** and a hex literal starts with **0x**.
- The literal `010` equals decimal **8** because of the leading-zero **octal** rule.

Data Structures - Deeper Nuances

- The total number of pointers in a doubly linked list of n nodes is **2n** (prev and next).
- A circular singly linked list has **no** NULL pointer, since the tail points to the **head**.
- A stack implemented with a single linked list does push/pop at the **head** in **O(1)**.
- A queue implemented with a linked list keeps **2** pointers: **front** and **rear**.
- Converting infix to postfix uses an **operator stack** of size up to **O(n)**.
- Evaluating an n-token postfix expression uses a stack of at most **O(n)** operands.
- A min-priority-queue extract-min costs **O(log n)** using a **min-heap**.
- The number of leaf nodes in a perfect binary tree of height h is **2^h**.
- The number of internal nodes in a perfect binary tree of height h is **2^h - 1**.
- The total nodes in a perfect binary tree of height h is **2^(h+1) - 1**.
- A complete binary tree with n nodes has height **floor(log2 n)**.
- The last internal node of a heap of size n is at index **(n/2) - 1** (0-based).
- Heapify starts from index **(n/2)-1** down to **0** for build-heap.
- The minimum number of nodes in an AVL tree of height h follows **N(h)=N(h-1)+N(h-2)+1**.
- An AVL tree's height is at most about **1.44 log2(n)**.
- The worst-case height of a red-black tree is at most **2 log2(n+1)**.
- A B-tree of order m has at most **m** children and at least **ceil(m/2)** children per internal node.
- The maximum degree of any vertex in a simple graph with n vertices is **n-1**.
- The number of edges in a complete directed graph on n vertices is **n(n-1)**.
- A connected undirected graph has at least **n-1** edges.
- A graph with more than **n-1** edges (n vertices, connected) must contain a **cycle**.
- BFS on an adjacency matrix costs **O(V^2)** while on a list it costs **O(V+E)**.
- The visited array in BFS/DFS prevents **infinite loops** on **cyclic** graphs.
- DFS on a graph can be implemented iteratively using an explicit **stack**.
- A spanning tree of a connected graph with n vertices has exactly **n-1** edges.

Hashing and Complexity - Deeper Nuances

- With separate chaining, the average search cost is **O(1 + alpha)**.
- A perfect hash function gives **O(1)** worst-case lookup with **no** collisions.
- The expected number of probes for successful linear-probing search is about **(1/2)(1 + 1/(1-alpha))**.
- A typical resize threshold for open addressing is load factor **0.7**.
- Doubling the table on rehash gives **amortised O(1)** insertion.
- The best, average, and worst-case array access are all **O(1)**.
- Binary search needs a **sorted** array and runs in **O(log n)**.
- Linear search runs in **O(n)** worst case and **O(1)** best case.
- The space complexity of recursion equals **O(maximum recursion depth)**.
- A balanced binary tree of n nodes has height **O(log n)**.
- A degenerate (skewed) tree of n nodes has height **n-1** and **O(n)** operations.
- The time to build a heap of n elements is **O(n)**, tighter than the naive **O(n log n)**.
- Inserting n elements one-by-one into a heap costs **O(n log n)** total.
- The lookup in a sorted linked list is still **O(n)** because there is **no** random access.
- A doubly linked list deletion given the node pointer is **O(1)**.
- A singly linked list deletion of the last node is **O(n)** without a previous pointer.

C Output-Prediction Essentials

- The expression `printf("%d", 5 + 2 * 3)` prints **11** due to precedence.
- The expression `printf("%d", 10 % 3)` prints **1**.
- The expression `printf("%d", -10 % 3)` prints **-1** (sign follows dividend).
- The expression `printf("%d", 5 > 3)` prints **1** (true).
- The expression `printf("%c", 65)` prints **A**.
- The expression `printf("%d", 'A')` prints **65**.
- The expression `printf("%d", sizeof(int))` typically prints **4**.
- The expression `int a[5]; printf("%d", sizeof(a))` typically prints **20**.
- For `int a[5]`, the value of `sizeof(a)/sizeof(a[0])` is **5**.
- The expression `printf("%d", 1 << 3)` prints **8**.
- The expression `printf("%d", 8 >> 2)` prints **2**.
- The expression `printf("%d", 5 & 3)` prints **1**.
- The expression `printf("%d", 5 | 3)` prints **7**.
- The expression `printf("%d", 5 ^ 3)` prints **6**.
- The expression `int x=5; printf("%d %d", x++, x)` is **undefined** due to unsequenced reads.
- The expression `int i=0; printf("%d", i++ + ++i)` is **undefined behaviour**.
- The loop `for(i=0;i<5;i++)` runs the body **5** times.
- The loop `for(i=0;i<=5;i++)` runs the body **6** times.
- The loop `i=0; while(i++ < 3)` runs the body **3** times.
- A `printf` returns the **count** of characters written.

Pointer and Array Output Nuances

- For `int a[]={1,2,3}; int *p=a;` the value `*(p+1)` is **2**.
- For `int a[]={1,2,3};` the value `*(a+2)` is **3**.
- For `int a[]={10,20};` the value `p[1]` where p=a is **20**.
- The expression `a[i]` is internally ***(a+i)** for any array a.
- Incrementing `char *p` by 1 advances **1** byte.
- Incrementing `double *p` by 1 advances **8** bytes.
- For pointer p to int, p+3 advances **12** bytes on a 4-byte-int system.
- A 2D array element `a[1][2]` in `int a[3][4]` is at offset **(1*4+2)=6** elements from base.
- The number of elements in `int a[3][4]` is **12**.
- The expression `&a[2] - &a[0]` equals **2** (element difference).

String Function Nuances

- `strcpy(dest, src)` copies until and including the **'\0'**.
- `strcat(dest, src)` appends src starting at dest's **'\0'**.
- `strcmp("abc","abd")` returns a **negative** value.
- `strcmp("abc","abc")` returns **0**.
- `strlen("")` returns **0** for an empty string.
- `strncpy` copies at most **n** characters and may not add **'\0'**.
- `strchr(s,c)` returns a **pointer** to the first occurrence or **NULL**.

- `strstr(s,sub)` returns a **pointer** to the first substring match or **NULL**.
- Reversing a string of length n needs **n/2** swaps in **O(n)**.
- Checking a palindrome compares index i with index **n-1-i**.

Recursion Tracing Nuances

- `factorial(5)` returns **120** after **6** calls (including base case).
- `factorial(0)` immediately returns **1** via the base case.
- `fib(0)` is **0** and `fib(1)` is **1** by convention.
- `fib(5)` evaluates to **5** using **15** total naive calls.
- `gcd(48,18)` evaluates to **6**.
- `power(2,10)` evaluates to **1024**.
- Hanoi with 4 disks needs **15** moves.
- A recursion with depth d uses **d** stack frames.
- Tail recursion can be converted into a **loop** to use **O(1)** stack.
- Mutual recursion involves at least **2** functions calling each other.

Stack/Queue Application Nuances

- The postfix of infix `A+B*C` is **ABC*+**.
- The prefix of infix `A+B*C` is **+A*BC**.
- The postfix of `(A+B)*C` is **AB+C***.
- Evaluating postfix `23*4+` gives **10**.
- Evaluating postfix `53-2*` gives **4**.
- A balanced expression `{[()]}` is **balanced** with stack empty at end.
- An expression `([)]` is **not balanced** due to interleaving.
- Function recursion uses the **system call stack** (LIFO).
- A circular queue of size 5 holds at most **4** elements with one slot unused.
- BFS of a graph uses a **FIFO** queue while DFS uses a **LIFO** stack.

Tree Traversal Nuances

- For BST inserting 50,30,70,20,40, the inorder is **20,30,40,50,70**.
- The preorder of a BST always starts with the **root**.
- The postorder of a BST always ends with the **root**.
- Level-order of a complete tree visits nodes in **array-index** order.
- A tree with preorder ABC and inorder BAC has root **A**.
- The first element of preorder is always the **root**.
- The last element of postorder is always the **root**.
- The middle element split in inorder separates **left** and **right** subtrees.
- A binary tree with only **inorder** given has **multiple** possible shapes.
- The height of a perfect binary tree with 7 nodes is **2**.

BST and Heap Sequence Nuances

- Inserting 1,2,3,4,5 into a plain BST yields a **right-skewed** tree of height **4**.

- Deleting the root of a 2-child BST uses the **inorder successor**.
- The minimum of a BST is found by going **left** repeatedly.
- A max-heap's largest element is always at the **root** (index 0).
- A min-heap's smallest element is always at the **root** (index 0).
- After inserting into a heap, **sift-up** restores the heap property.
- After deleting the root, the **last** element moves to root then **sift-down**.
- The parent of index 6 (0-based) is index **2**.
- The children of index 2 (0-based) are indices **5** and **6**.
- Building a max-heap from [3,1,2] gives root **3**.

Graph and Hashing Nuances

- A graph with 4 vertices that is complete (K4) has **6** edges.
- A directed complete graph on 4 vertices has **12** edges.
- A tree with 6 nodes has exactly **5** edges.
- An adjacency matrix for an undirected graph is **symmetric**.
- The diagonal of a simple-graph adjacency matrix is all **0** (no self-loops).
- Hashing 25 with `h(k)=k mod 10` gives index **5**.
- Hashing keys 12,22 with `mod 10` causes a **collision** at index **2**.
- Linear probing resolves a collision by checking the **next** slot.
- A load factor of 0.75 means the table is **75%** full.
- Doubling table size keeps amortised insert at **O(1)**.

Final Rapid-Fire Facts

- The default integer promotion target for char/short is **int**.
- A NULL pointer compares equal to integer constant **0**.
- The number of arguments to `calloc` is **2**.
- The number of arguments to `realloc` is **2**.
- The maximum children of a binary tree node is **2**.
- The number of NULL links in a binary tree with n nodes is **n+1**.
- The number of edges in any tree with n nodes is **n-1**.
- The base case of factorial is at n = **0**.
- The minimum moves for Tower of Hanoi with n disks is **2^n - 1**.
- The worst-case time of quicksort-like skewed BST operations is **O(n)**.
- The average BST height for random insertions is **O(log n)**.
- The complexity to find the height of a binary tree is **O(n)**.
- The complexity to count nodes in a binary tree is **O(n)**.
- The complexity to search a balanced BST is **O(log n)**.
- The complexity to access the k-th linked-list node is **O(k)**.
- The complexity of enqueue and dequeue in a queue is **O(1)**.
- The complexity of push and pop in a stack is **O(1)**.
- The complexity of binary search is **O(log n)**.
- The complexity of linear search is **O(n)**.
- The complexity of heap sort is **O(n log n)**.
- The complexity of building a heap is **O(n)**.

- The complexity of inorder traversal is **O(n)**.
- The complexity of BFS/DFS with adjacency list is **O(V+E)**.
- The space used by an adjacency matrix is **O(V^2)**.
- The space used by an adjacency list is **O(V+E)**.
- The number of comparisons in binary search of n elements is at most **ceil(log2(n+1))**.
- The recurrence for merge-style divide and conquer is **T(n)=2T(n/2)+O(n)**.
- A perfect binary tree of height 3 has **15** nodes.
- A full binary tree always has an **odd** number of nodes.
- The number of distinct BSTs with 3 keys is the Catalan number **5**.
- The inorder successor of a node with a right child is the right subtree's **minimum**.
- The inorder predecessor of a node with a left child is the left subtree's **maximum**.
- A queue is best for **level-order** traversal of a tree.
- A stack is implicitly used by **recursive** tree traversals.
- The data structure behind a priority queue is typically a **binary heap**.
- The data structure used for **LRU cache** is often a **hash map + doubly linked list**.

Extended C Concept Drills

- The keyword `auto` is the **default** storage class and is rarely written explicitly.
- A `register` request is only a **hint** that the compiler may **ignore**.
- The number of bytes returned by `sizeof(double)` is typically **8**.
- The number of bytes returned by `sizeof(char)` is always **1**.
- The number of bytes in `sizeof("A")` is **2** (the char plus '\0').
- A function with no parameters should be declared `f(void)` to disable **implicit args**.
- The `extern` keyword is used to share a global across multiple **translation units**.
- The lifetime of a `static` local variable is the **entire program**.
- The scope of a `static` global variable is the **file** (internal linkage).
- An array cannot be **returned** directly from a function; return a **pointer** instead.
- A const-qualified variable must be **initialised** at declaration.
- The bitwise complement of 0 (`~0`) in an int is **-1** (all bits set).
- The expression `x & (x-1)` clears the **lowest set bit** of x.
- The expression `x & -x` isolates the **lowest set bit** of x.
- A power of two has exactly **1** bit set, so `x & (x-1)` equals **0**.
- The number of set bits is the **population count** (Hamming weight).
- A left rotation differs from a left shift by wrapping the **overflow** bits.
- The ternary chained `a?b:c?d:e` associates **right** as `a?b:(c?d:e)`.
- The comma in a function call separates **arguments**, not the comma operator.
- A `break` inside nested loops exits only the **innermost** loop.

Extended Data-Structure Drills

- A stack overflow in recursion occurs when frames exceed the **stack size** limit.
- A queue underflow occurs when **dequeue** is called on an **empty** queue.
- A stack overflow (array) occurs when **push** is called on a **full** stack.
- The minimum number of stacks needed to implement a queue is **2**.
- The minimum number of queues needed to implement a stack is **2** (or **1** with rotation).

- A deque can simulate both a **stack** and a **queue**.
- An array-based stack has top initialised to **-1** when empty.
- A linked-list traversal visits each node exactly **once** in **O(n)**.
- Inserting at the head of a singly linked list updates the **head** pointer.
- Deleting the head of a singly linked list is **O(1)**.
- Inserting at the tail with a tail pointer is **O(1)**; without it is **O(n)**.
- A sentinel/dummy node simplifies linked-list **insertion/deletion** edge cases.
- Reversing a doubly linked list swaps each node's **prev** and **next**.
- Merging two sorted linked lists is **O(n+m)**.
- Detecting a loop with Floyd's algorithm uses **2** pointers at speeds **1** and **2**.
- The meeting point in Floyd's cycle detection helps find the **loop start**.

Extended Tree Drills

- The number of leaves equals the number of **degree-2** nodes plus **1** in a full binary tree.
- A binary tree of height h has at least **h+1** nodes (skewed).
- A binary tree of height h has at most **2^(h+1)-1** nodes (perfect).
- The diameter of a tree is the longest **path** between two nodes.
- The number of binary trees with 4 nodes is the Catalan number **14**.
- An expression tree's leaves are **operands** and internal nodes are **operators**.
- Evaluating an expression tree uses a **postorder** traversal.
- A threaded binary tree replaces NULL links with **inorder** threads.
- The successor in a threaded tree is reached without a **stack**.
- A BST allows **floor** and **ceil** queries in **O(h)**.
- The k-th smallest element in a BST uses **inorder** traversal.
- An AVL insertion may trigger at most **1** (single or double) rotation.
- An AVL deletion may trigger up to **O(log n)** rotations along the path.
- A red-black tree insertion needs at most **2** rotations.
- The black-height counts black nodes on any **root-to-leaf** path.

Extended Graph and Hashing Drills

- A self-loop adds **2** to a vertex's degree in an undirected graph.
- A multigraph allows **multiple** edges between the same pair of vertices.
- A simple graph has **no** self-loops and **no** multiple edges.
- The maximum edges in a simple undirected graph on n vertices is **n(n-1)/2**.
- A strongly connected directed graph has a path between **every** ordered pair.
- Topological sort applies only to a **DAG**.
- A topological order can be produced by **DFS** finish times or **Kahn's** algorithm.
- An adjacency matrix uses **1** bit/entry minimum for unweighted graphs.
- Counting connected components needs a BFS/DFS from each **unvisited** vertex.
- A hash collision occurs when **2** keys map to the **same** index.
- A good hash function minimises **collisions** and spreads keys **uniformly**.
- Chaining degrades to **O(n)** when all keys hash to **one** bucket.
- Open addressing requires **deletion** to use **tombstones** (lazy deletion).
- The probe sequence in double hashing depends on a **second** hash function.

- A prime table size reduces clustering in **quadratic** probing.

Final Complexity Cheat Drills

- The best case of insertion sort is **O(n)** and worst is **O(n^2)**.
- The complexity of accessing a hash entry on average is **O(1)**.
- The complexity of the worst-case hash lookup is **O(n)**.
- The complexity of finding min in a min-heap is **O(1)**.
- The complexity of finding max in a min-heap is **O(n)**.
- The complexity of decrease-key in a binary heap is **O(log n)**.
- The complexity of merging two binary heaps naively is **O(n)**.
- The space complexity of an iterative traversal with a stack is **O(h)**.
- The space complexity of level-order traversal is **O(width)**.
- The complexity of reversing a linked list is **O(n)** time, **O(1)** space.
- The complexity of array rotation by k is **O(n)**.
- The complexity of checking balanced parentheses is **O(n)** time, **O(n)** space.
- The complexity of converting infix to postfix is **O(n)**.
- The complexity of evaluating a postfix expression is **O(n)**.
- The complexity of constructing a tree from traversals is **O(n)**.

Supplementary Exam One-Liners

- The number of ways to fully parenthesise n+1 factors is the Catalan number **C(n)**.
- The smallest unsigned type guaranteed by C is **unsigned char** of **1** byte.
- The expression `!0` evaluates to **1** and `!5` evaluates to **0**.
- The logical operators in C return only **0** or **1**.
- A compound literal like `(int){5}` creates an **unnamed** object.
- The maximum index accessible in `int a[10]` is **9**.
- The number of dimensions in `int a[2][3][4]` is **3** with **24** elements.
- A jagged 2D structure is best built with an **array of pointers**.
- The function to release dynamic memory is **free**.
- A memory leak does **not** crash immediately but exhausts the **heap** over time.
- The default visibility of a C global variable is **external linkage**.
- A header file should contain **declarations**, not **definitions**, to avoid duplicate symbols.
- The `#pragma once` directive is an alternative to **include guards**.
- The maximum number of children in an n-ary tree node is **n**.
- A binary tree is a special case of an n-ary tree with n = **2**.
- A forest is a collection of **disjoint** trees.
- Removing the root of a forest's tree may create **multiple** subtrees.
- The number of edges in a forest with n nodes and k trees is **n-k**.
- A stack-based DFS visits a graph's vertices in **depth-first** order.
- A queue-based BFS visits a graph's vertices in **breadth-first** order.
- The complexity of inserting into a sorted array maintaining order is **O(n)**.
- The complexity of deleting from a hash table on average is **O(1)**.
- A circular buffer is implemented as a **circular queue**.
- The two pointers in a doubly linked list are **prev** and **next**.

- The total comparisons to find both min and max can be done in **3n/2** comparisons.
- The lower bound for comparison-based sorting is **O(n log n)**.
- A binary heap stored in an array needs **no** explicit pointers.
- The height of a binary heap with n nodes is **floor(log2 n)**.
- The leaves of a heap occupy roughly the last **n/2** array positions.
- A priority queue's two main operations are **insert** and **extract-min/max**.
- The data structure giving O(1) average and O(n) worst lookup is the **hash table**.
- The data structure giving guaranteed O(log n) operations is the **balanced BST (AVL)**.
- The data structure giving O(1) LIFO operations is the **stack**.
- The data structure giving O(1) FIFO operations is the **queue**.